

Guia de Instalação e Uso — extrato.py

Este documento descreve o processo de instalação, configuração e utilização da aplicação `extrato.py`, responsável por coletar dados de uma API e persisti-los em banco de dados MySQL de forma controlada e segura.

A aplicação foi projetada para operar em ambiente de produção com execução diária, evitando sobrecarga no servidor de origem ao realizar consultas sempre de forma sequencial, um dia por vez. Além disso, permite reprocessamento histórico para realinhamento de dados.

Visão geral da aplicação

A aplicação executa as seguintes etapas:

- recebe uma data opcional como parâmetro de entrada;
 - define o período de processamento entre a data informada e a data atual;
 - gera uma sequência de datas;
 - realiza consultas à API, uma por vez, para cada dia do período;
 - valida a resposta da API;
 - grava os dados no banco MySQL utilizando chave única por registro;
 - atualiza registros existentes sem duplicação;
 - em caso de erro em um dia específico, realiza rollback apenas daquele dia e continua o processamento dos demais.
-

Pré-requisitos

Antes da instalação, é necessário garantir que o ambiente possua:

- Python 3 instalado;
 - gerenciador de pacotes pip;
 - acesso a um banco MySQL ou MariaDB;
 - permissões de rede para acesso à API;
 - permissões de escrita no banco de dados configurado.
-

Instalação no Linux

Para sistemas Debian, Ubuntu ou derivados:

```
apt update  
apt install -y python3 python3-pip python3-venv
```

Para sistemas AlmaLinux, Rocky ou CentOS:

```
dnf install -y python3 python3-pip
```

Instalação no Windows

1. Baixar o Python em:

<https://www.python.org/downloads/>

2. Durante a instalação, marcar a opção:

Add Python to PATH

3. Validar a instalação:

```
python --version
```

Instalação das dependências

Crie um arquivo chamado `requirements.txt` com o seguinte conteúdo:

```
requests>=2.31.0  
mysql-connector-python>=8.0.33
```

Linux

```
python3 -m pip install -r requirements.txt --break-system-packages
```

Windows

```
python -m pip install -r requirements.txt
```

Validação das dependências

Após a instalação, execute:

Linux

```
python3 -c "import requests; import mysql.connector; print('dependencias ok')"
```

Windows

```
python -c "import requests; import mysql.connector; print('dependencias ok')"
```

Validação da conexão com o MySQL

Recomenda-se validar a conexão com o banco de dados antes da execução:

Linux

```
python3 - <<'PY'
import mysql.connector

conn = mysql.connector.connect(
    host="127.0.0.1",
    user="root",
    password="SUA_SENHA",
    database="SEU_BANCO"
)
print("mysql ok")
conn.close()
PY
```

Windows

```
python -c "import mysql.connector;
conn=mysql.connector.connect(host='127.0.0.1',user='root',password='SUA_SENHA',d
atabase='SEU_BANCO'); print('mysql ok'); conn.close()"
```

Configuração da aplicação

No arquivo `extrato.py`, configure os dados de conexão com o banco:

```
DB_CONFIG = {  
    "host": "127.0.0.1",  
    "user": "root",  
    "password": "SUA_SENHA",  
    "database": "SEU_BANCO",  
    "charset": "utf8mb4"  
}
```

Configure também os parâmetros de acesso à API:

```
API_USER = "SEU_USUARIO"  
API_PWD = "SUA_SENHA"
```

Execução da aplicação

Execução padrão (dia atual)

```
python3 extrato.py
```

ou no Windows:

```
python extrato.py
```

Neste modo, a aplicação processa apenas a data atual.

Execução com data específica

```
python3 extrato.py 2026-03-30
```

Neste modo, a aplicação processa:

- a data informada;
 - todos os dias subsequentes;
 - até a data atual.
-

Comportamento de processamento

Para cada data:

- a aplicação realiza uma única chamada à API;
 - valida o retorno recebido;
 - percorre os registros do bloco CSV;
 - grava cada registro no banco utilizando chave única;
 - utiliza atualização automática em caso de registros já existentes;
 - realiza commit ao final do dia;
 - em caso de erro, executa rollback apenas do dia afetado.
-

Execução automática diária

Linux (cron)

Editar o crontab:

```
crontab -e
```

Adicionar a linha:

```
0 2 * * * python3 /caminho/extrato.py >> /var/log/extrato.log 2>&1
```

Executa diariamente às 02:00.

Windows (Agendador de Tarefas)

1. Abrir o Agendador de Tarefas;
 2. Criar nova tarefa;
 3. Configurar:
 - Programa: caminho do Python;
 - Argumento: caminho do `extrato.py`.
-

Reprocessamento de dados

Para realinhamento ou correção de dados históricos:

```
python3 extrato.py 2026-01-01
```

A aplicação irá:

- reprocessar todos os dias desde a data informada até hoje;
 - atualizar registros existentes sem duplicação.
-

Monitoramento

Durante a execução, a aplicação apresenta mensagens como:

```
Processando 2026-03-30 ...  
Data 2026-03-30: csv=8000 gravados=8000
```

Em caso de erro:

```
Erro ao processar 2026-03-31: ...
```

Boas práticas

- executar a aplicação uma vez ao dia;
 - utilizar reprocessamento apenas quando necessário;
 - monitorar logs regularmente;
 - evitar execuções simultâneas;
 - garantir estabilidade do banco e da API antes da execução.
-

Estrutura recomendada

```
/opt/extrato_app/  
├─ extrato.py  
├─ requirements.txt  
└─ logs/
```

Resumo de uso

Ação	Comando
Executar dia atual	<code>python extrato.py</code>
Reprocessar período	<code>python extrato.py 2026-03-30</code>
Instalar dependências	<code>pip install -r requirements.txt</code>

Este processo garante uma coleta controlada, segura e consistente dos dados, com capacidade de recuperação e alinhamento histórico sem impacto significativo na infraestrutura de origem.